

EMPLOYEE TRACKING SYSTEM

Capstone Project Report

Technology Stack: Java 17, Spring Boot, Spring Security, Thymeleaf, MySQL, Bootstrap 5

Project Type: Full-Stack Web Application

Submission Date: 05 April 2026

INDEX

CHAPTER 1 :	
INTRODUCTION	3
CHAPTER 2 :	
LITERATURE REVIEW	5
CHAPTER 3 :	
METHODOLOGY	7
CHAPTER 4 :	
DESIGN AND IMPLEMENTATION	10
• System Flow Diagram	10
• Activity Lifecycle Diagram	11
• Use Case Diagram	12
• Data Flow Diagram (DFD)	13
• Class Diagram	14
• Sequence Diagram	15
CHAPTER 5 :	
BACKEND CODE IMPLEMENTATION	16
• Configuration Layer	16
• Model Layer	18
• Repository, Service & Controller Layers	21
CHAPTER 6 :	
TESTING	28
CHAPTER 7 :	
RESULT AND APPLICATIONS	30
CHAPTER 8 :	
CONCLUSION AND FUTURE SCOPE	34
CHAPTER 9 :	
REFERENCES AND BIBLIOGRAPHY	36

CHAPTER 1 : INTRODUCTION

1.1 Abstract

This capstone report presents the design and implementation of an **Employee Tracking System (ETS)**, a role-based web application developed using Spring Boot and MySQL. The system centralizes employee data management, attendance recording, leave processing, and user administration. It provides dedicated dashboards for administrators and employees, ensuring secure access and intuitive workflows.

The project follows layered architecture principles with clear separation across controller, service, repository, model, and view layers. Authentication and authorization are implemented with Spring Security and BCrypt password hashing, making the solution suitable for real-world organizational use.

1.2 Introduction

Organizations require a reliable digital system to maintain employee records and day-to-day HR operations. Manual or spreadsheet-based methods create inconsistencies, delay approvals, and limit reporting capabilities. ETS is designed to provide a compact, maintainable, and secure system that streamlines employee and attendance operations while improving visibility for administrators.

The system serves two types of users: administrators who manage all HR operations, and employees who interact with their own attendance and leave data. All interactions are protected by Spring Security, ensuring that no user can access resources beyond their role.

1.3 Problem Statement

Traditional employee record systems often suffer from scattered data, weak access control, and process delays in attendance and leave workflows. The project addresses these issues by developing a centralized portal with role-specific privileges, form validation, and controlled approval pipelines.

- Lack of centralized digital records for employee information.
- Manual and error-prone attendance tracking processes.
- No formal leave request and approval mechanism.
- Absence of role-based access separation for HR staff and employees.

1.4 Objectives

- Develop a secure web-based employee information management platform.
- Implement attendance check-in/check-out with historical records.
- Provide a leave request and approval workflow.
- Enable administrators to manage departments, users, and employee profiles.
- Ensure usability through a responsive UI and clean navigation.

1.5 Scope of the Project

The current scope includes employee CRUD, department management, attendance processing, leave application and approval, and user enable/disable operations. The solution supports two roles: ADMIN and EMPLOYEE. It targets educational and small-to-medium enterprise scenarios and can be extended for enterprise reporting.

CHAPTER 2 : LITERATURE REVIEW

A review of existing employee management and tracking systems reveals a range of solutions from enterprise platforms like SAP HCM and Oracle HRMS to open-source alternatives. This chapter examines the key findings from literature that informed the design decisions for ETS.

2.1 Existing Systems and Gaps

System / Study	Approach	Limitation
SAP HCM	Enterprise HR suite with full payroll integration	High cost, complex setup, not suitable for small organizations
Traditional Spreadsheet Systems	Excel-based manual records	No access control, prone to errors, no workflow automation
Open-source HRMS (OrangeHRM)	Modular HR management	Complex configuration, limited customization for developers
Custom PHP Applications	Lightweight web-based HR	Lack of security best practices, no standardized architecture

2.2 Relevant Technologies Studied

The following technologies were reviewed and selected based on industry adoption, documentation quality, and suitability for the project scale:

- **Spring Boot:** Provides auto-configuration, embedded server, and a production-ready foundation for Java web applications.
- **Spring Security:** A comprehensive security framework supporting authentication, authorization, and CSRF protection out of the box.
- **Spring Data JPA + Hibernate:** Simplifies data access through repository abstraction and ORM mapping, reducing boilerplate SQL.
- **Thymeleaf:** A server-side Java template engine with natural HTML syntax, ideal for MVC-based applications.
- **Bootstrap 5:** A front-end framework enabling rapid development of responsive, mobile-friendly UI components.
- **MySQL:** A widely-used open-source relational database with strong JPA/Hibernate integration.

2.3 Key Design Principles from Literature

- **Role-Based Access Control (RBAC):** Separation of admin and employee privileges as a core security principle.
- **Layered Architecture:** Clear separation of presentation, business logic, and data access layers for maintainability.
- **DTO Pattern:** Using Data Transfer Objects to decouple API boundaries from entity models.
- **Repository Pattern:** Abstracting data store operations behind repository interfaces for testability.

CHAPTER 3 : METHODOLOGY

This chapter describes the development approach, technology choices, system architecture, database schema, and security methodology used to build ETS.

3.1 Development Approach

The project follows an iterative development methodology. Each module (Authentication, Employee, Department, Attendance, Leave) was designed, implemented, and tested independently before integration. The MVC pattern with Spring Boot provides a clean separation of concerns throughout.

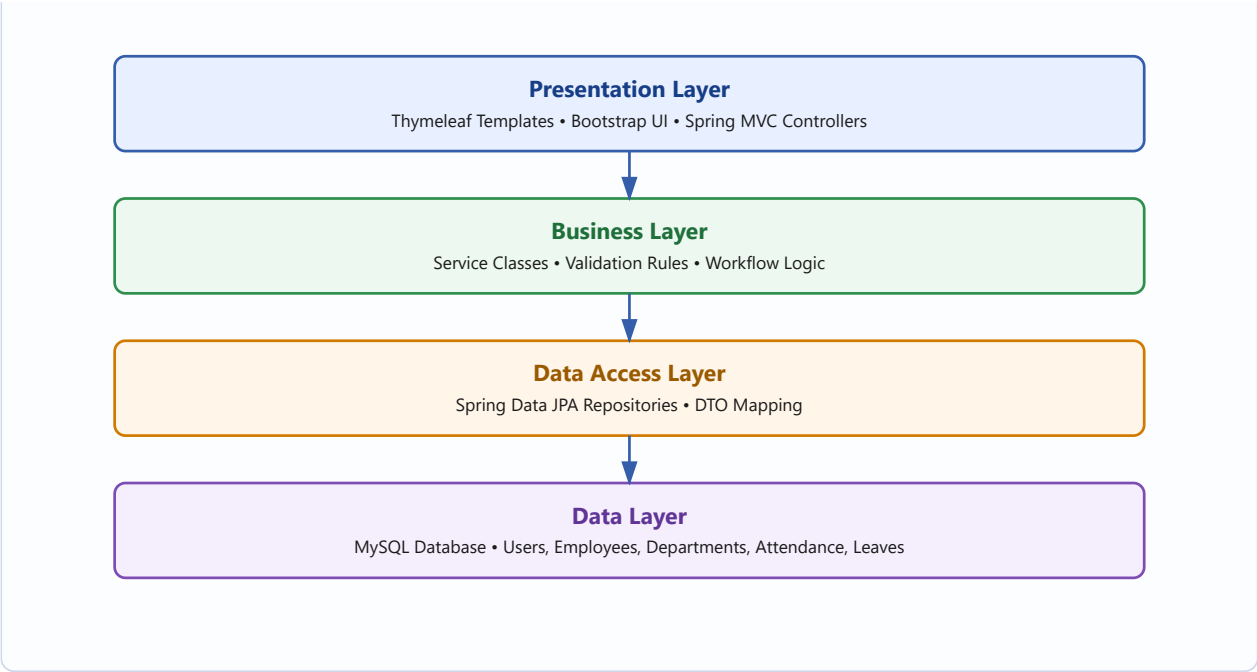
3.2 Technology Stack

Layer	Technology	Purpose
Language	Java 17+	Core application development
Framework	Spring Boot 4.0.3	Application runtime and dependency management
Security	Spring Security + BCrypt	Authentication, authorization, password protection
Persistence	Spring Data JPA + Hibernate	ORM and repository abstraction
Database	MySQL 8.0+	Relational data storage
Frontend	Thymeleaf + Bootstrap 5	Server-side rendering and responsive UI
Build Tool	Maven	Build, test, package lifecycle

3.3 System Architecture

ETS follows a four-layer architecture. Controllers process HTTP requests, services contain business logic, repositories handle data access, and entity models represent domain objects. DTOs are used for controlled data transfer across layers.

Figure 3.1: Layered System Architecture



3.4 Database Design

The database is normalized around five main entities with foreign key relationships enforcing referential integrity.

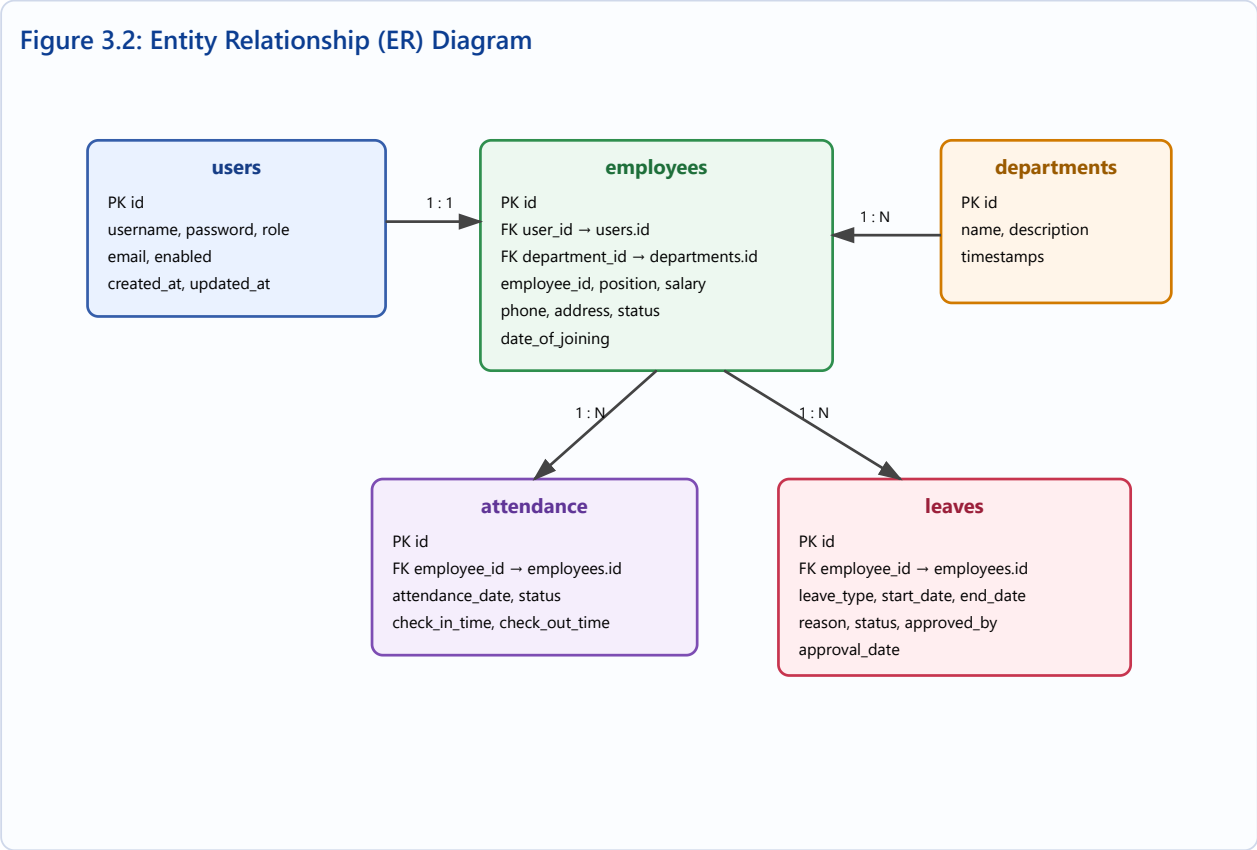
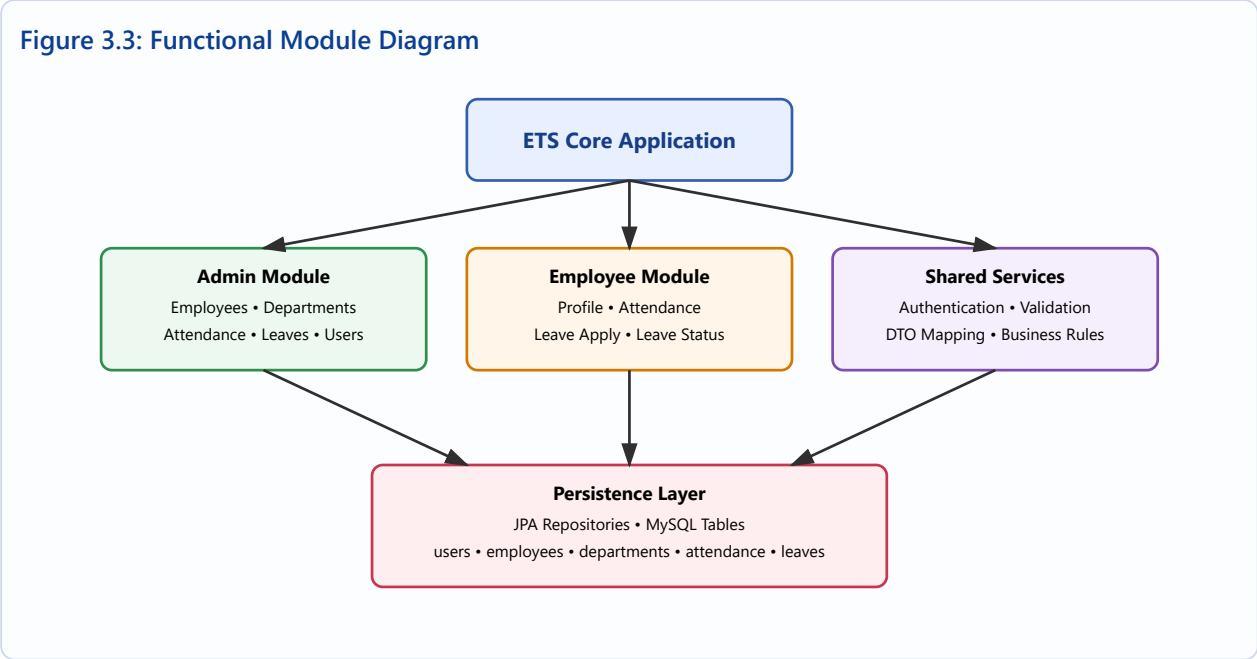


Table	Description	Key Relationships
users	Application authentication and role information	1-to-1 with employees

Table	Description	Key Relationships
employees	Employee personal and professional profile	Many-to-1 with departments
departments	Department metadata	Referenced by employees
attendance	Daily attendance log with check-in/check-out	Many-to-1 with employees
leaves	Leave applications and decision history	Many-to-1 with employees

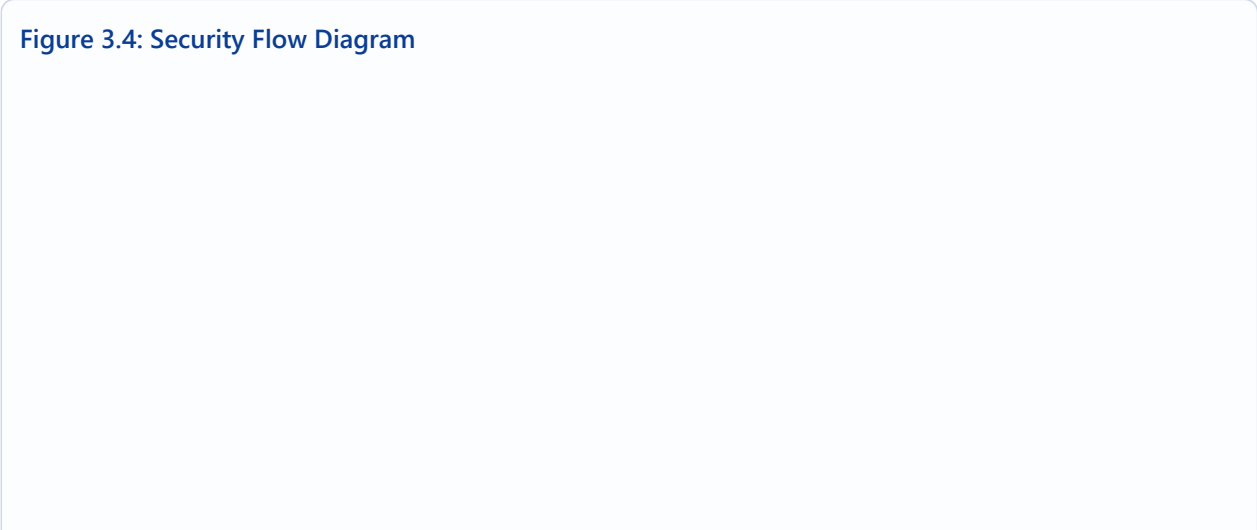
3.5 Module Implementation

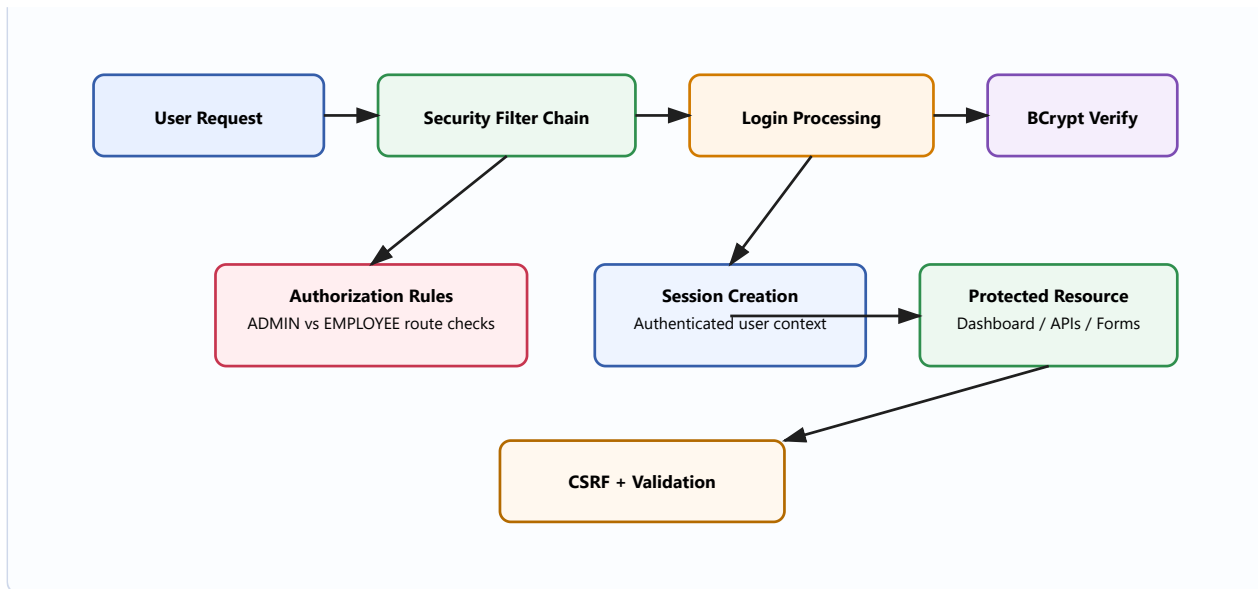


Admin Module: Dashboard, employee CRUD, department management, attendance monitoring, leave approvals, user account control.

Employee Module: Personal dashboard, attendance check-in/check-out, attendance history, leave application, leave status, profile view.

3.6 Security Design





- Role-based access control using ADMIN and EMPLOYEE authorities.
- Spring Security login/logout flow with protected routes.
- BCrypt hashing for credential storage.
- CSRF protection and secure form submissions.
- Validation constraints for input quality and integrity.

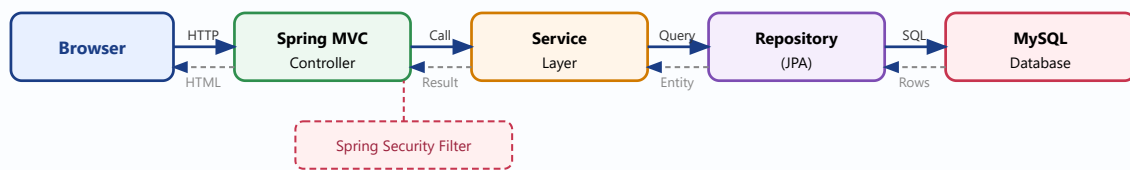
CHAPTER 4 : DESIGN AND IMPLEMENTATION

This chapter presents the detailed design artifacts of the Employee Tracking System, illustrating system flow, actor interactions, data movement, class relationships, and runtime message exchanges.

11.1 System Flow Diagram

The system flow shows the end-to-end request processing path from browser to database and back.

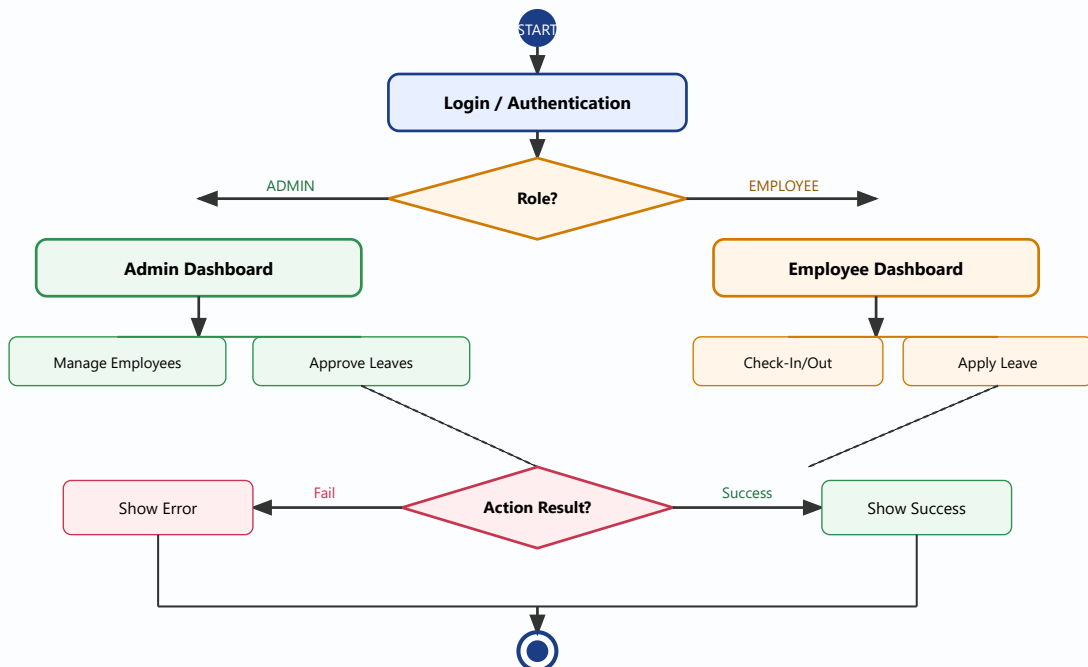
Figure 11.1: System Flow Diagram



11.2 Activity Lifecycle Diagram

The activity lifecycle diagram shows the main user workflows — login routing, attendance management, and leave processing — from start to completion.

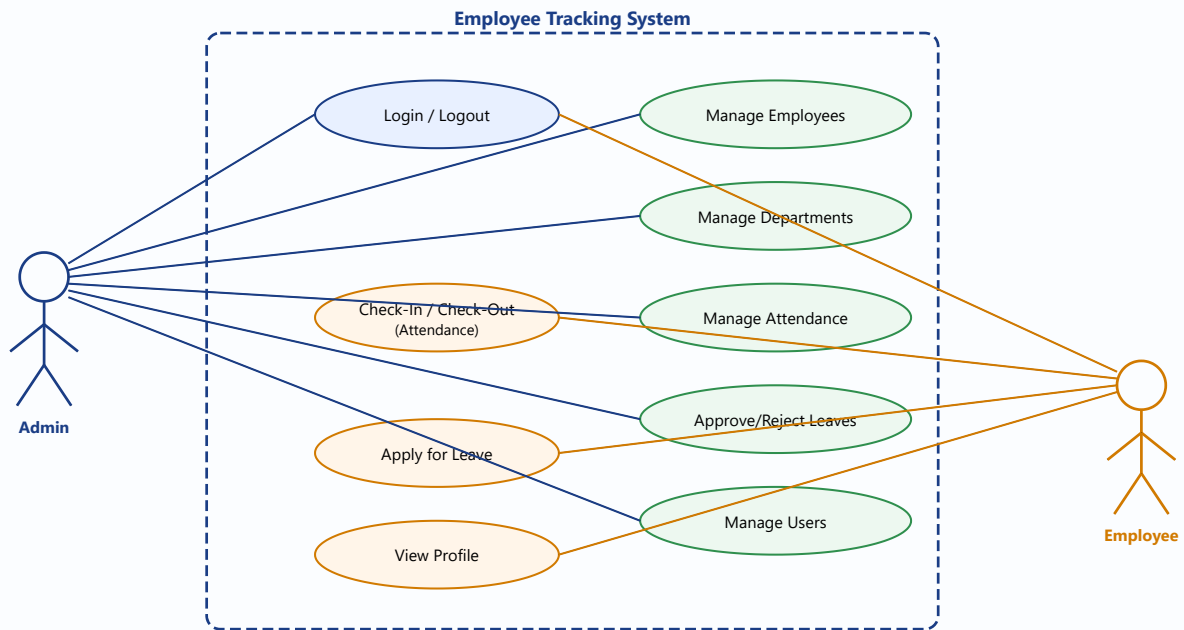
Figure 11.2: Activity Lifecycle Diagram



11.3 Use Case Diagram

The use case diagram identifies all actors and their interactions with the system's core functionalities.

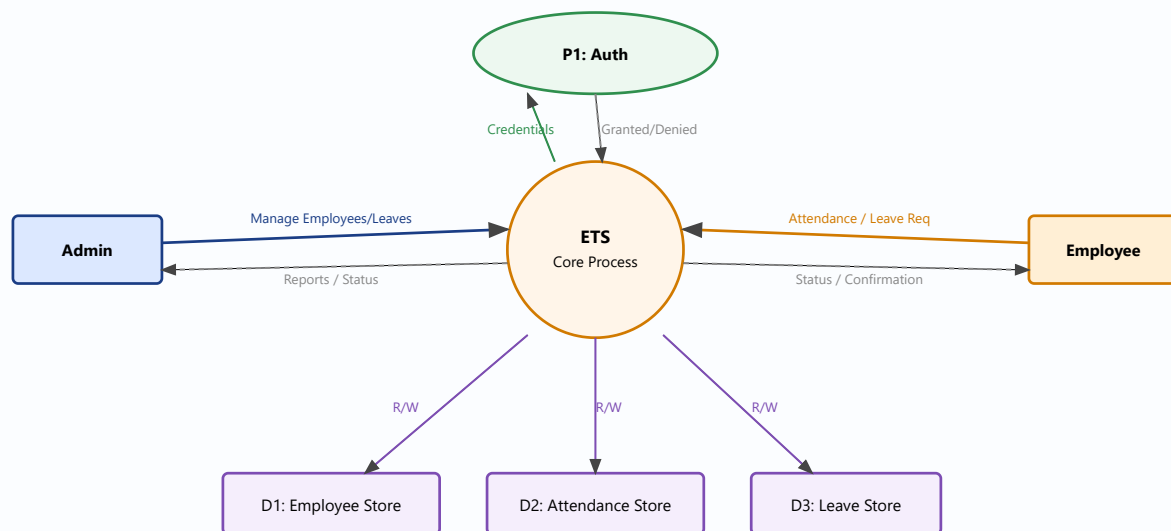
Figure 11.3: Use Case Diagram



11.4 Data Flow Diagram (DFD)

The DFD shows how data moves between external entities, processes, and data stores within the system.

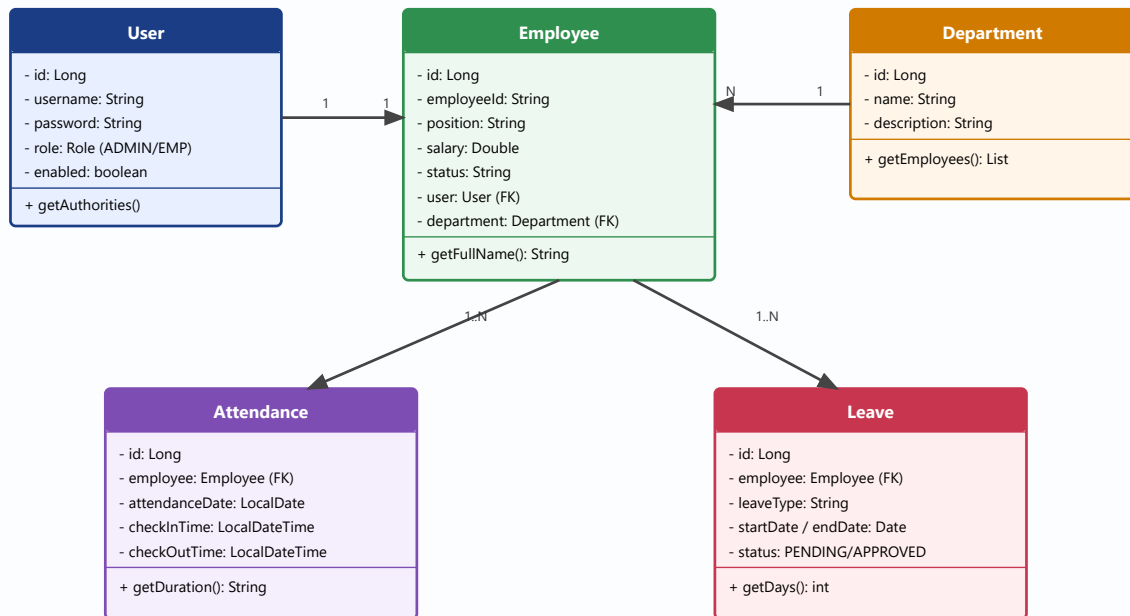
Figure 11.4: Level-1 Data Flow Diagram



11.5 Class Diagram

The class diagram shows the main domain entities, their attributes, and relationships as modeled in the JPA layer.

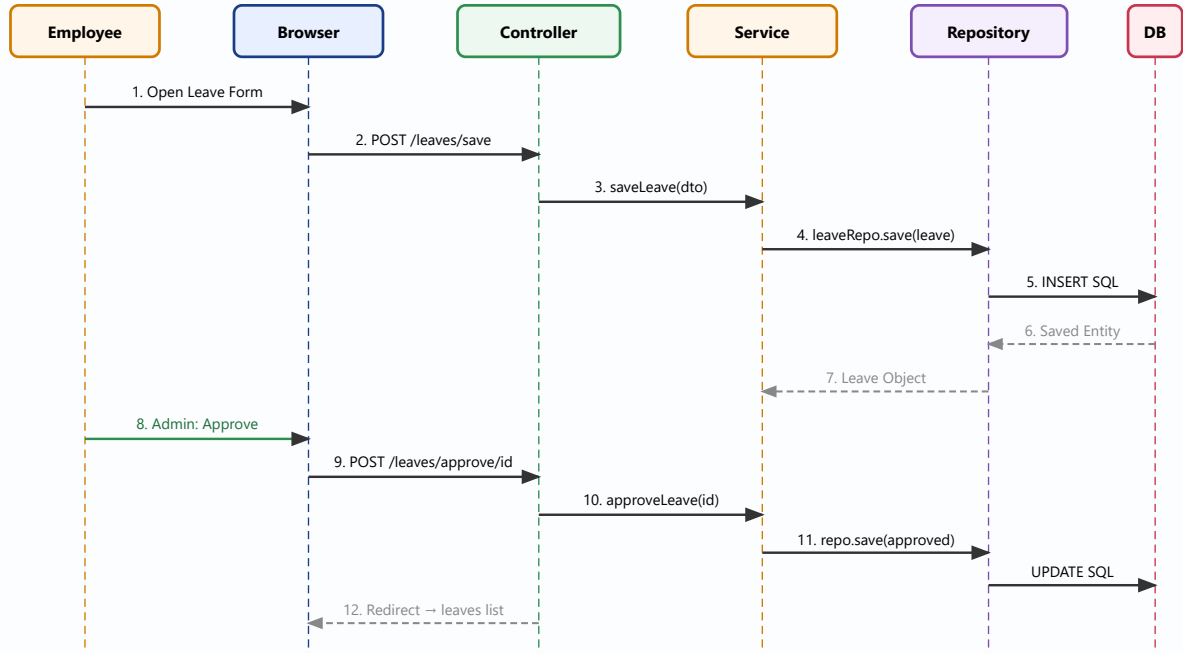
Figure 11.5: Class Diagram



11.6 Sequence Diagram

The sequence diagram shows the message exchange between components during the Leave Application flow.

Figure 11.6: Sequence Diagram — Leave Application & Approval Flow



CHAPTER 5 : BACKEND CODE IMPLEMENTATION

This chapter presents the complete backend source code of the Employee Tracking System, organized by architectural layer: application entry, configuration, model, DTO, repository, service, and controller. All code is written in Java 17 using Spring Boot 4.x conventions.

5.1 Application Entry Point

The main class bootstraps the Spring Boot application context.

EtsApplication.java — com.data.ets

```
package com.data.ets;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EtsApplication {

    public static void main(String[] args) {
        SpringApplication.run(EtsApplication.class, args);
    }

}
```

5.2 Configuration Layer

The configuration package contains Spring Security setup, custom authentication logic, and the data seeder that initializes sample records on first startup.

SecurityConfig.java — com.data.ets.config

```

package com.data.ets.config;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final UserDetailsService userDetailsService;
    private final CustomAuthenticationSuccessHandler successHandler;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider daoAuthenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/", "/login", "/css/**", "/js/**", "/images/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/employee/**").hasRole("EMPLOYEE")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .loginProcessingUrl("/login")
                .successHandler(successHandler)
                .failureUrl("/login?error")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .permitAll()
            )
            .exceptionHandling(exception -> exception
                .accessDeniedPage("/access-denied")
            );

        return http.build();
    }
}

```

CustomUserDetailsService.java — com.data.ets.config

```

package com.data.ets.config;

import com.data.ets.model.User;
import com.data.ets.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found with username: " + username));

        List<GrantedAuthority> authorities = Collections.singletonList(
            new SimpleGrantedAuthority("ROLE_" + user.getRole().toString())
        );

        return org.springframework.security.core.userdetails.User.builder()
            .username(user.getUsername())
            .password(user.getPassword())
            .authorities(authorities)
            .accountExpired(false)
            .accountLocked(false)
            .credentialsExpired(false)
            .disabled(!user.getEnabled())
            .build();
    }
}

```

CustomAuthenticationSuccessHandler.java — com.data.ets.config

```

package com.data.ets.config;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class CustomAuthenticationSuccessHandler extends SimpleUrlAuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
                                       Authentication authentication) throws IOException, ServletException {

        // Check user's role and redirect accordingly
        for (GrantedAuthority authority : authentication.getAuthorities()) {
            if (authority.getAuthority().equals("ROLE_ADMIN")) {
                setDefaultTargetUrl("/admin/dashboard");
                break;
            } else if (authority.getAuthority().equals("ROLE_EMPLOYEE")) {
                setDefaultTargetUrl("/employee/dashboard");
                break;
            }
        }

        super.onAuthenticationSuccess(request, response, authentication);
    }
}

```

DataInitializer.java — com.data.ets.config

```

package com.data.ets.config;

import com.data.ets.model.*;
import com.data.ets.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    private final UserRepository userRepository;
    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;
    private final AttendanceRepository attendanceRepository;
    private final LeaveRepository leaveRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    public void run(String... args) throws Exception {
        // Check if data already exists
        if (userRepository.count() > 0) {
            return;
        }

        // Create Departments
        Department itDept = Department.builder()
            .name("Information Technology")
            .description("IT Department")
            .build();
        itDept = departmentRepository.save(itDept);

        Department hrDept = Department.builder()
            .name("Human Resources")
            .description("HR Department")
            .build();
        hrDept = departmentRepository.save(hrDept);

        Department salesDept = Department.builder()
            .name("Sales")
            .description("Sales Department")
            .build();
        salesDept = departmentRepository.save(salesDept);

        // Create Admin User
        User adminUser = User.builder()
            .username("admin")
            .email("admin@ets.com")
            .password(passwordEncoder.encode("admin123"))
            .firstName("Admin")
            .lastName("User")
            .role(Role.ADMIN)
            .enabled(true)
            .build();
        userRepository.save(adminUser);

        // Create Employee Users
        User emp1User = User.builder()
            .username("emp1")
            .email("emp1@ets.com")
            .password(passwordEncoder.encode("emp123"))
            .firstName("John")
            .lastName("Doe")
            .role(Role.EMPLOYEE)
            .enabled(true)
            .build();
        emp1User = userRepository.save(emp1User);

        User emp2User = User.builder()
            .username("emp2")
            .email("emp2@ets.com")
            .password(passwordEncoder.encode("emp123"))
            .firstName("Jane")
            .lastName("Smith")
            .role(Role.EMPLOYEE)
            .enabled(true)
            .build();
        emp2User = userRepository.save(emp2User);

        User emp3User = User.builder()
            .username("emp3")
            .email("emp3@ets.com")
            .password(passwordEncoder.encode("emp123"))
            .firstName("Mike")

```

```

        .lastName("Johnson")
        .role(Role.EMPLOYEE)
        .enabled(true)
        .build();
emp3User = userRepository.save(emp3User);

// Create Employees
Employee emp1 = Employee.builder()
    .user(emp1User)
    .employeeId("EMP001")
    .phoneNumber("9876543210")
    .dateOfBirth(LocalDate.of(1995, 5, 15))
    .gender("Male")
    .department(itDept)
    .position("Software Engineer")
    .salary(50000.0)
    .dateOfJoining(LocalDate.of(2020, 1, 15))
    .address("123 Main St")
    .city("New York")
    .state("NY")
    .zipCode("10001")
    .status("ACTIVE")
    .build();
emp1 = employeeRepository.save(emp1);

Employee emp2 = Employee.builder()
    .user(emp2User)
    .employeeId("EMP002")
    .phoneNumber("9876543211")
    .dateOfBirth(LocalDate.of(1998, 8, 20))
    .gender("Female")
    .department(hrDept)
    .position("HR Manager")
    .salary(45000.0)
    .dateOfJoining(LocalDate.of(2019, 6, 1))
    .address("456 Oak Ave")
    .city("Los Angeles")
    .state("CA")
    .zipCode("90001")
    .status("ACTIVE")
    .build();
emp2 = employeeRepository.save(emp2);

Employee emp3 = Employee.builder()
    .user(emp3User)
    .employeeId("EMP003")
    .phoneNumber("9876543212")
    .dateOfBirth(LocalDate.of(1992, 3, 10))
    .gender("Male")
    .department(salesDept)
    .position("Sales Executive")
    .salary(40000.0)
    .dateOfJoining(LocalDate.of(2021, 3, 15))
    .address("789 Pine Rd")
    .city("Chicago")
    .state("IL")
    .zipCode("60601")
    .status("ACTIVE")
    .build();
emp3 = employeeRepository.save(emp3);

// Create Sample Attendance Records
Attendance att1 = Attendance.builder()
    .employee(emp1)
    .attendanceDate(LocalDate.now())
    .checkInTime(LocalDateTime.now().withHour(9).withMinute(0))
    .checkOutTime(LocalDateTime.now().withHour(17).withMinute(30))
    .status("PRESENT")
    .remarks("Regular")
    .build();
attendanceRepository.save(att1);

Attendance att2 = Attendance.builder()
    .employee(emp2)
    .attendanceDate(LocalDate.now())
    .checkInTime(LocalDateTime.now().withHour(9).withMinute(15))
    .checkOutTime(LocalDateTime.now().withHour(17).withMinute(45))
    .status("PRESENT")
    .remarks("Regular")
    .build();
attendanceRepository.save(att2);

// Create Sample Leave Requests
Leave leave1 = Leave.builder()
    .employee(emp1)
    .leaveType("SICK")
    .startDate(LocalDate.now().plusDays(1))
    .endDate(LocalDate.now().plusDays(2))
    .reason("Medical Appointment")
    .status("PENDING")

```



```
        .build();
        leaveRepository.save(leave1);

        Leave leave2 = Leave.builder()
            .employee(emp2)
            .leaveType("CASUAL")
            .startDate(LocalDate.now().plusDays(5))
            .endDate(LocalDate.now().plusDays(7))
            .reason("Family vacation")
            .status("APPROVED")
            .approvedBy("Admin")
            .approvalDate(LocalDateTime.now())
            .build();
        leaveRepository.save(leave2);

        System.out.println("Sample data initialized successfully!");
    }
}
```

5.3 Model Layer

JPA entity classes mapped to MySQL tables. Lombok annotations reduce boilerplate; `@PrePersist` / `@PreUpdate` handle audit timestamps automatically.

Role.java — com.data.ets.model

```
package com.data.ets.model;

public enum Role {
    ADMIN,
    EMPLOYEE
}
```

User.java — com.data.ets.model

```
package com.data.ets.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role;

    @Column(nullable = false)
    private Boolean enabled = true;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Employee.java — com.data.ets.model

```

package com.data.ets.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "employees")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "employee_id", unique = true, nullable = false)
    private String employeeId;

    @Column(name = "phone_number")
    private String phoneNumber;

    @Column(name = "date_of_birth")
    private LocalDate dateOfBirth;

    @Column(name = "gender")
    private String gender;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    @Column(name = "position")
    private String position;

    @Column(name = "salary")
    private Double salary;

    @Column(name = "date_of_joining")
    private LocalDate dateOfJoining;

    @Column(name = "address")
    private String address;

    @Column(name = "city")
    private String city;

    @Column(name = "state")
    private String state;

    @Column(name = "zip_code")
    private String zipCode;

    @Column(name = "status")
    private String status = "ACTIVE";

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}

```

Department.java — com.data.ets.model

```
package com.data.ets.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;
import java.util.List;

@Entity
@Table(name = "departments")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @Column
    private String description;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Attendance.java — com.data.ets.model

```
package com.data.ets.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "attendance")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Attendance {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Column(name = "attendance_date", nullable = false)
    private LocalDate attendanceDate;

    @Column(name = "check_in_time")
    private LocalDateTime checkInTime;

    @Column(name = "check_out_time")
    private LocalDateTime checkOutTime;

    @Column(name = "status")
    private String status; // PRESENT, ABSENT, LATE, ON_LEAVE

    @Column(name = "remarks")
    private String remarks;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }
}
```

Leave.java — com.data.ets.model

```

package com.data.ets.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "leaves")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Leave {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Column(name = "leave_type", nullable = false)
    private String leaveType; // SICK, CASUAL, EARNED, UNPAID

    @Column(name = "start_date", nullable = false)
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false)
    private LocalDate endDate;

    @Column(name = "reason")
    private String reason;

    @Column(name = "status")
    private String status = "PENDING"; // PENDING, APPROVED, REJECTED

    @Column(name = "approved_by")
    private String approvedBy;

    @Column(name = "approval_date")
    private LocalDateTime approvalDate;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}

```

5.4 Data Transfer Objects (DTO)

DTOs decouple the API and service layers from JPA entity internals, carrying only the fields needed by each view or operation.

EmployeeDTO.java — com.data.ets.dto

```
package com.data.ets.dto;

import lombok.*;

import java.time.LocalDate;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class EmployeeDTO {
    private Long id;
    private String username;
    private String email;
    private String firstName;
    private String lastName;
    private String employeeId;
    private String phoneNumber;
    private LocalDate dateOfBirth;
    private String gender;
    private String departmentName;
    private Long departmentId;
    private String position;
    private Double salary;
    private LocalDate dateOfJoining;
    private String address;
    private String city;
    private String state;
    private String zipCode;
    private String status;
}
```

DepartmentDTO.java — com.data.ets.dto

```
package com.data.ets.dto;

import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DepartmentDTO {
    private Long id;
    private String name;
    private String description;
}
```

AttendanceDTO.java — com.data.ets.dto

```
package com.data.ets.dto;

import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class AttendanceDTO {
    private Long id;
    private Long employeeId;
    private String employeeName;
    private LocalDate attendanceDate;
    private LocalDateTime checkInTime;
    private LocalDateTime checkOutTime;
    private String status;
    private String remarks;
}
```

LeaveDTO.java — com.data.ets.dto

```

package com.data.ets.dto;

import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class LeaveDTO {
    private Long id;
    private Long employeeId;
    private String employeeName;
    private String leaveType;
    private LocalDate startDate;
    private LocalDate endDate;
    private String reason;
    private String status;
    private String approvedBy;
    private LocalDateTime approvalDate;
}

```

UserDTO.java — com.data.ets.dto

```

package com.data.ets.dto;

import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserDTO {
    private Long id;
    private String username;
    private String email;
    private String firstName;
    private String lastName;
    private String role;
    private Boolean enabled;
}

```

5.5 Repository Layer

Spring Data JPA repositories provide CRUD operations and custom query methods with zero boilerplate SQL.

UserRepository.java — com.data.ets.repository

```

package com.data.ets.repository;

import com.data.ets.model.User;
import com.data.ets.model.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
    List<User> findByRole(Role role);
    List<User> findByEnabled(Boolean enabled);
}

```

EmployeeRepository.java — com.data.ets.repository


```

package com.data.ets.repository;

import com.data.ets.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Optional<Employee> findById(String employeeId);
    Optional<Employee> findById(Long userId);
    List<Employee> findByDepartmentId(Long departmentId);
    List<Employee> findByStatus(String status);
}

```

DepartmentRepository.java — com.data.ets.repository

```

package com.data.ets.repository;

import com.data.ets.model.Department;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface DepartmentRepository extends JpaRepository<Department, Long> {
    Optional<Department> findByName(String name);
}

```

AttendanceRepository.java — com.data.ets.repository

```

package com.data.ets.repository;

import com.data.ets.model.Attendance;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface AttendanceRepository extends JpaRepository<Attendance, Long> {
    List<Attendance> findById(Long employeeId);
    List<Attendance> findByAttendanceDate(LocalDate attendanceDate);
    List<Attendance> findByIdAndAttendanceDateBetween(Long employeeId, LocalDate startDate, LocalDate endDate);
}

```

LeaveRepository.java — com.data.ets.repository

```

package com.data.ets.repository;

import com.data.ets.model.Leave;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface LeaveRepository extends JpaRepository<Leave, Long> {
    List<Leave> findById(Long employeeId);
    List<Leave> findByStatus(String status);
    List<Leave> findByIdAndStatus(Long employeeId, String status);
    List<Leave> findByStartDateBetween(LocalDate startDate, LocalDate endDate);
}

```

5.6 Service Layer

Service classes encapsulate all business logic, including DTO conversion, workflow validation, and approval state transitions.

UserService.java — com.data.ets.service

```

package com.data.ets.service;

import com.data.ets.dto.UserDTO;
import com.data.ets.model.Role;
import com.data.ets.model.User;
import com.data.ets.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public User createUser(String username, String email, String password, String firstName, String lastName, Role role) {
        User user = User.builder()
            .username(username)
            .email(email)
            .password(passwordEncoder.encode(password))
            .firstName(firstName)
            .lastName(lastName)
            .role(role)
            .enabled(true)
            .build();
        return userRepository.save(user);
    }

    public Optional<User> findByUsername(String username) {
        return userRepository.findByUsername(username);
    }

    public Optional<User> findByEmail(String email) {
        return userRepository.findByEmail(email);
    }

    public Optional<User> findById(Long id) {
        return userRepository.findById(id);
    }

    public List<User> findAll() {
        return userRepository.findAll();
    }

    public List<User> findByRole(Role role) {
        return userRepository.findByRole(role);
    }

    public List<UserDTO> getAllUsers() {
        return userRepository.findAll().stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public List<UserDTO> getAllEmployees() {
        return userRepository.findByRole(Role.EMPLOYEE).stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public User updateUser(Long id, User updatedUser) {
        return userRepository.findById(id).map(user -> {
            user.setEmail(updatedUser.getEmail());
            user.setFirstName(updatedUser.getFirstName());
            user.setLastName(updatedUser.getLastName());
            user.setEnabled(updatedUser.isEnabled());
            return userRepository.save(user);
        }).orElseThrow(() -> new RuntimeException("User not found"));
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }

    public void enableUser(Long id) {
        userRepository.findById(id).ifPresent(user -> {
            user.setEnabled(true);
            userRepository.save(user);
        });
    }

    public void disableUser(Long id) {
        userRepository.findById(id).ifPresent(user -> {
            user.setEnabled(false);

```

```
        userRepository.save(user);
    });
}

private UserDTO convertToDTO(User user) {
    return UserDTO.builder()
        .id(user.getId())
        .username(user.getUsername())
        .email(user.getEmail())
        .firstName(user.getFirstName())
        .lastName(user.getLastName())
        .role(user.getRole().toString())
        .enabled(user.getEnabled())
        .build();
}
}
```

EmployeeService.java — com.data.ets.service

```

package com.data.ets.service;

import com.data.ets.dto.EmployeeDTO;
import com.data.ets.model.Employee;
import com.data.ets.model.User;
import com.data.ets.repository.EmployeeRepository;
import com.data.ets.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class EmployeeService {
    private final EmployeeRepository employeeRepository;
    private final UserRepository userRepository;

    public Employee createEmployee(Employee employee) {
        return employeeRepository.save(employee);
    }

    public Optional<Employee> findById(Long id) {
        return employeeRepository.findById(id);
    }

    public Optional<Employee> findByEmployeeId(String employeeId) {
        return employeeRepository.findByEmployeeId(employeeId);
    }

    public Optional<Employee> findByUserId(Long userId) {
        return employeeRepository.findByUserId(userId);
    }

    public List<Employee> findAll() {
        return employeeRepository.findAll();
    }

    public List<Employee> findByDepartmentId(Long departmentId) {
        return employeeRepository.findByDepartmentId(departmentId);
    }

    public List<Employee> findByStatus(String status) {
        return employeeRepository.findByStatus(status);
    }

    public List<EmployeeDTO> getAllEmployees() {
        return employeeRepository.findAll().stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public EmployeeDTO getEmployeeDTO(Long id) {
        return employeeRepository.findById(id)
            .map(this::convertToDTO)
            .orElseThrow(() -> new RuntimeException("Employee not found"));
    }

    public Employee updateEmployee(Long id, Employee updatedEmployee) {
        return employeeRepository.findById(id).map(employee -> {
            employee.setPhoneNumber(updatedEmployee.getPhoneNumber());
            employee.setDateOfBirth(updatedEmployee.getDateOfBirth());
            employee.setGender(updatedEmployee.getGender());
            employee.setDepartment(updatedEmployee.getDepartment());
            employee.setPosition(updatedEmployee.getPosition());
            employee.setSalary(updatedEmployee.getSalary());
            employee.setAddress(updatedEmployee.getAddress());
            employee.setCity(updatedEmployee.getCity());
            employee.setState(updatedEmployee.getState());
            employee.setZipCode(updatedEmployee.getZipCode());
            employee.setStatus(updatedEmployee.getStatus());
            return employeeRepository.save(employee);
        }).orElseThrow(() -> new RuntimeException("Employee not found"));
    }

    public void deleteEmployee(Long id) {
        employeeRepository.deleteById(id);
    }

    public EmployeeDTO convertToDTO(Employee employee) {
        return EmployeeDTO.builder()
            .id(employee.getId())
            .username(employee.getUser().getUsername())
            .email(employee.getUser().getEmail())
            .firstName(employee.getUser().getFirstName())
            .lastName(employee.getUser().getLastName())
            .employeeId(employee.getEmployeeId())

```

```

        .phoneNumber(employee.getPhoneNumber())
        .dateOfBirth(employee.getDateOfBirth())
        .gender(employee.getGender())
        .departmentName(employee.getDepartment() != null ? employee.getDepartment().getName() : null)
        .departmentId(employee.getDepartment() != null ? employee.getDepartment().getId() : null)
        .position(employee.getPosition())
        .salary(employee.getSalary())
        .dateOfJoining(employee.getDateOfJoining())
        .address(employee.getAddress())
        .city(employee.getCity())
        .state(employee.getState())
        .zipCode(employee.getZipCode())
        .status(employee.getStatus())
        .build();
    }
}

```

DepartmentService.java — com.data.ets.service

```

package com.data.ets.service;

import com.data.ets.dto.DepartmentDTO;
import com.data.ets.model.Department;
import com.data.ets.repository.DepartmentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class DepartmentService {
    private final DepartmentRepository departmentRepository;

    public Department createDepartment(Department department) {
        return departmentRepository.save(department);
    }

    public Optional<Department> findById(Long id) {
        return departmentRepository.findById(id);
    }

    public Optional<Department> findByName(String name) {
        return departmentRepository.findByName(name);
    }

    public List<Department> findAll() {
        return departmentRepository.findAll();
    }

    public List<DepartmentDTO> getAllDepartments() {
        return departmentRepository.findAll().stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public Department updateDepartment(Long id, Department updatedDepartment) {
        return departmentRepository.findById(id).map(department -> {
            department.setName(updatedDepartment.getName());
            department.setDescription(updatedDepartment.getDescription());
            return departmentRepository.save(department);
        }).orElseThrow(() -> new RuntimeException("Department not found"));
    }

    public void deleteDepartment(Long id) {
        departmentRepository.deleteById(id);
    }

    private DepartmentDTO convertToDTO(Department department) {
        return DepartmentDTO.builder()
            .id(department.getId())
            .name(department.getName())
            .description(department.getDescription())
            .build();
    }
}

```

AttendanceService.java — com.data.ets.service

```

package com.data.ets.service;

import com.data.ets.dto.AttendanceDTO;
import com.data.ets.model.Attendance;
import com.data.ets.model.Employee;
import com.data.ets.repository.AttendanceRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class AttendanceService {
    private final AttendanceRepository attendanceRepository;

    public Attendance checkIn(Long employeeId) {
        Employee employee = new Employee();
        employee.setId(employeeId);
        Attendance attendance = new Attendance();
        attendance.setEmployee(employee);
        attendance.setAttendanceDate(LocalDate.now());
        attendance.setCheckInTime(LocalDateTime.now());
        attendance.setStatus("PRESENT");
        return attendanceRepository.save(attendance);
    }

    public Attendance checkOut(Long attendanceId) {
        return attendanceRepository.findById(attendanceId).map(attendance -> {
            attendance.setCheckOutTime(LocalDateTime.now());
            return attendanceRepository.save(attendance);
        }).orElseThrow(() -> new RuntimeException("Attendance record not found"));
    }

    public Attendance createAttendance(Attendance attendance) {
        return attendanceRepository.save(attendance);
    }

    public Optional<Attendance> findById(Long id) {
        return attendanceRepository.findById(id);
    }

    public List<Attendance> findByEmployeeId(Long employeeId) {
        return attendanceRepository.findByEmployeeId(employeeId);
    }

    public List<Attendance> findByAttendanceDate(LocalDate attendanceDate) {
        return attendanceRepository.findByAttendanceDate(attendanceDate);
    }

    public List<Attendance> findByEmployeeIdAndDateRange(Long employeeId, LocalDate startDate, LocalDate endDate) {
        return attendanceRepository.findByEmployeeIdAndAttendanceDateBetween(employeeId, startDate, endDate);
    }

    public List<AttendanceDTO> getAllAttendance() {
        return attendanceRepository.findAll().stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public List<AttendanceDTO> getEmployeeAttendance(Long employeeId) {
        return attendanceRepository.findByEmployeeId(employeeId).stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public Attendance updateAttendance(Long id, Attendance updatedAttendance) {
        return attendanceRepository.findById(id).map(attendance -> {
            attendance.setStatus(updatedAttendance.getStatus());
            attendance.setRemarks(updatedAttendance.getRemarks());
            return attendanceRepository.save(attendance);
        }).orElseThrow(() -> new RuntimeException("Attendance record not found"));
    }

    public void deleteAttendance(Long id) {
        attendanceRepository.deleteById(id);
    }

    private AttendanceDTO convertToDTO(Attendance attendance) {
        return AttendanceDTO.builder()
            .id(attendance.getId())
            .employeeId(attendance.getEmployee().getId())
            .employeeName(attendance.getEmployee().getUser().getFirstName() + " " + attendance.getEmployee().getUser().getLastName())
            .attendanceDate(attendance.getAttendanceDate())
            .checkInTime(attendance.getCheckInTime())

```

```
        .checkOutTime(attendance.getCheckOutTime())  
        .status(attendance.getStatus())  
        .remarks(attendance.getRemarks())  
        .build();  
    }  
}
```

LeaveService.java — com.data.ets.service

```

package com.data.ets.service;

import com.data.ets.dto.LeaveDTO;
import com.data.ets.model.Leave;
import com.data.ets.repository.LeaveRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class LeaveService {
    private final LeaveRepository leaveRepository;

    public Leave createLeave(Leave leave) {
        leave.setStatus("PENDING");
        return leaveRepository.save(leave);
    }

    public Optional<Leave> findById(Long id) {
        return leaveRepository.findById(id);
    }

    public List<Leave> findByEmployeeId(Long employeeId) {
        return leaveRepository.findByEmployeeId(employeeId);
    }

    public List<Leave> findByStatus(String status) {
        return leaveRepository.findByStatus(status);
    }

    public List<Leave> findPendingLeaves() {
        return leaveRepository.findByStatus("PENDING");
    }

    public List<Leave> findByEmployeeIdAndStatus(Long employeeId, String status) {
        return leaveRepository.findByEmployeeIdAndStatus(employeeId, status);
    }

    public List<LeaveDTO> getAllLeaves() {
        return leaveRepository.findAll().stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public List<LeaveDTO> getEmployeeLeaves(Long employeeId) {
        return leaveRepository.findByEmployeeId(employeeId).stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public List<LeaveDTO> getPendingLeaves() {
        return leaveRepository.findByStatus("PENDING").stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    public Leave approveLeave(Long id, String approvedBy) {
        return leaveRepository.findById(id).map(leave -> {
            leave.setStatus("APPROVED");
            leave.setApprovedBy(approvedBy);
            leave.setApprovalDate(LocalDate.now());
            return leaveRepository.save(leave);
        }).orElseThrow(() -> new RuntimeException("Leave request not found"));
    }

    public Leave rejectLeave(Long id, String approvedBy) {
        return leaveRepository.findById(id).map(leave -> {
            leave.setStatus("REJECTED");
            leave.setApprovedBy(approvedBy);
            leave.setApprovalDate(LocalDate.now());
            return leaveRepository.save(leave);
        }).orElseThrow(() -> new RuntimeException("Leave request not found"));
    }

    public Leave updateLeave(Long id, Leave updatedLeave) {
        return leaveRepository.findById(id).map(leave -> {
            leave.setLeaveType(updatedLeave.getLeaveType());
            leave.setStartDate(updatedLeave.getStartDate());
            leave.setEndDate(updatedLeave.getEndDate());
            leave.setReason(updatedLeave.getReason());
            return leaveRepository.save(leave);
        }).orElseThrow(() -> new RuntimeException("Leave request not found"));
    }
}

```



```

public void deleteLeave(Long id) {
    leaveRepository.deleteById(id);
}

private LeaveDTO convertToDTO(Leave leave) {
    return LeaveDTO.builder()
        .id(leave.getId())
        .employeeId(leave.getEmployee().getId())
        .employeeName(leave.getEmployee().getUser().getFirstName() + " " + leave.getEmployee().getUser().getLastName())
        .leaveType(leave.getLeaveType())
        .startDate(leave.getStartDate())
        .endDate(leave.getEndDate())
        .reason(leave.getReason())
        .status(leave.getStatus())
        .approvedBy(leave.getApprovedBy())
        .approvalDate(leave.getApprovalDate())
        .build();
}
}

```

5.7 Controller Layer

Spring MVC controllers handle HTTP routing, delegate to services, populate the Thymeleaf model, and return view names. All admin routes require `ROLE_ADMIN`; employee routes require `ROLE_EMPLOYEE`.

HomeController.java — com.data.ets.controller

```

package com.data.ets.controller;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        // Check if user is authenticated
        if (authentication != null && authentication.isAuthenticated() &&
            !authentication.getPrincipal().equals("anonymousUser")) {

            // Check user's role and redirect accordingly
            if (authentication.getAuthorities().stream()
                .anyMatch(auth -> auth.getAuthority().equals("ROLE_ADMIN"))) {
                return "redirect:/admin/dashboard";
            } else if (authentication.getAuthorities().stream()
                .anyMatch(auth -> auth.getAuthority().equals("ROLE_EMPLOYEE"))) {
                return "redirect:/employee/dashboard";
            }
        }

        return "redirect:/login";
    }

    @GetMapping("/login")
    public String login() {
        return "login";
    }

    @GetMapping("/access-denied")
    public String accessDenied() {
        return "access-denied";
    }
}

```

AdminController.java — com.data.ets.controller

```

package com.data.ets.controller;

import com.data.ets.dto.EmployeeDTO;
import com.data.ets.model.Department;
import com.data.ets.model.Employee;
import com.data.ets.model.Role;
import com.data.ets.model.User;
import com.data.ets.service.DepartmentService;
import com.data.ets.service.EmployeeService;
import com.data.ets.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Controller
@RequestMapping("/admin")
@RequiredArgsConstructor
public class AdminController {
    private final UserService userService;
    private final EmployeeService employeeService;
    private final DepartmentService departmentService;

    @GetMapping("/dashboard")
    public String dashboard(Model model) {
        model.addAttribute("totalEmployees", employeeService.findAll().size());
        model.addAttribute("totalUsers", userService.findAll().size());
        model.addAttribute("totalDepartments", departmentService.findAll().size());
        return "admin/dashboard";
    }

    // Employee Management
    @GetMapping("/employees")
    public String listEmployees(Model model) {
        List<EmployeeDTO> employees = employeeService.getAllEmployees();
        model.addAttribute("employees", employees);
        return "admin/employees/list";
    }

    @GetMapping("/employees/add")
    public String addEmployeeForm(Model model) {
        List<Department> departments = departmentService.findAll();
        model.addAttribute("departments", departments);
        model.addAttribute("roles", Role.values());
        return "admin/employees/add";
    }

    @PostMapping("/employees/save")
    public String saveEmployee(@RequestParam String username,
                               @RequestParam String email,
                               @RequestParam String password,
                               @RequestParam String firstName,
                               @RequestParam String lastName,
                               @RequestParam String employeeId,
                               @RequestParam String phoneNumber,
                               @RequestParam String position,
                               @RequestParam Double salary,
                               @RequestParam Long departmentId) {
        User user = userService.createUser(username, email, password, firstName, lastName, Role.EMPLOYEE);

        Department department = departmentService.findById(departmentId).orElse(null);
        Employee employee = Employee.builder()
            .user(user)
            .employeeId(employeeId)
            .phoneNumber(phoneNumber)
            .position(position)
            .salary(salary)
            .department(department)
            .status("ACTIVE")
            .build();

        employeeService.createEmployee(employee);
        return "redirect:/admin/employees";
    }

    @GetMapping("/employees/edit/{id}")
    public String editEmployeeForm(@PathVariable Long id, Model model) {
        EmployeeDTO employee = employeeService.getEmployeeDTO(id);
        List<Department> departments = departmentService.findAll();
        model.addAttribute("employee", employee);
        model.addAttribute("departments", departments);
        return "admin/employees/edit";
    }

    @PostMapping("/employees/update/{id}")
    public String updateEmployee(@PathVariable Long id,
                                @RequestParam String phoneNumber,

```

```

        @RequestParam String position,
        @RequestParam Double salary,
        @RequestParam Long departmentId) {
    Employee employee = employeeService.findById(id)
        .orElseThrow(() -> new RuntimeException("Employee not found"));

    employee.setPhoneNumber(phoneNumber);
    employee.setPosition(position);
    employee.setSalary(salary);
    employee.setDepartment(departmentService.findById(departmentId).orElse(null));

    employeeService.updateEmployee(id, employee);
    return "redirect:/admin/employees";
}

@GetMapping("/employees/delete/{id}")
public String deleteEmployee(@PathVariable Long id) {
    employeeService.deleteEmployee(id);
    return "redirect:/admin/employees";
}

// Department Management
@GetMapping("/departments")
public String listDepartments(Model model) {
    List<Department> departments = departmentService.findAll();
    model.addAttribute("departments", departments);
    return "admin/departments/list";
}

@GetMapping("/departments/add")
public String addDepartmentForm() {
    return "admin/departments/add";
}

@PostMapping("/departments/save")
public String saveDepartment(@RequestParam String name,
                             @RequestParam String description) {
    Department department = Department.builder()
        .name(name)
        .description(description)
        .build();
    departmentService.createDepartment(department);
    return "redirect:/admin/departments";
}

@GetMapping("/departments/edit/{id}")
public String editDepartmentForm(@PathVariable Long id, Model model) {
    Department department = departmentService.findById(id)
        .orElseThrow(() -> new RuntimeException("Department not found"));
    model.addAttribute("department", department);
    return "admin/departments/edit";
}

@PostMapping("/departments/update/{id}")
public String updateDepartment(@PathVariable Long id,
                              @RequestParam String name,
                              @RequestParam String description) {
    Department department = Department.builder()
        .name(name)
        .description(description)
        .build();
    departmentService.updateDepartment(id, department);
    return "redirect:/admin/departments";
}

@GetMapping("/departments/delete/{id}")
public String deleteDepartment(@PathVariable Long id) {
    departmentService.deleteDepartment(id);
    return "redirect:/admin/departments";
}

// User Management
@GetMapping("/users")
public String listUsers(Model model) {
    List<User> users = userService.findAll();
    model.addAttribute("users", users);
    return "admin/users/list";
}

@GetMapping("/users/disable/{id}")
public String disableUser(@PathVariable Long id) {
    userService.disableUser(id);
    return "redirect:/admin/users";
}

@GetMapping("/users/enable/{id}")
public String enableUser(@PathVariable Long id) {
    userService.enableUser(id);
    return "redirect:/admin/users";
}

```

```

    }
}

```

AttendanceController.java — com.data.ets.controller

```

package com.data.ets.controller;

import com.data.ets.dto.AttendanceDTO;
import com.data.ets.dto.LeaveDTO;
import com.data.ets.model.Attendance;
import com.data.ets.service.AttendanceService;
import com.data.ets.service.LeaveService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@Controller
@RequestMapping("/admin/attendance")
@RequiredArgsConstructor
public class AttendanceController {
    private final AttendanceService attendanceService;

    @GetMapping
    public String listAttendance(Model model) {
        List<AttendanceDTO> attendanceList = attendanceService.getAllAttendance();
        model.addAttribute("attendanceList", attendanceList);
        return "admin/attendance/list";
    }

    @GetMapping("/add")
    public String addAttendanceForm() {
        return "admin/attendance/add";
    }

    @PostMapping("/save")
    public String saveAttendance(@RequestParam Long employeeId,
                                @RequestParam LocalDate attendanceDate,
                                @RequestParam String status,
                                @RequestParam String remarks) {
        Attendance attendance = Attendance.builder()
            .attendanceDate(attendanceDate)
            .status(status)
            .remarks(remarks)
            .build();
        attendanceService.createAttendance(attendance);
        return "redirect:/admin/attendance";
    }

    @GetMapping("/edit/{id}")
    public String editAttendanceForm(@PathVariable Long id, Model model) {
        Attendance attendance = attendanceService.findById(id)
            .orElseThrow(() -> new RuntimeException("Attendance not found"));
        model.addAttribute("attendance", attendance);
        return "admin/attendance/edit";
    }

    @PostMapping("/update/{id}")
    public String updateAttendance(@PathVariable Long id,
                                   @RequestParam String status,
                                   @RequestParam String remarks) {
        Attendance attendance = Attendance.builder()
            .status(status)
            .remarks(remarks)
            .build();
        attendanceService.updateAttendance(id, attendance);
        return "redirect:/admin/attendance";
    }

    @GetMapping("/delete/{id}")
    public String deleteAttendance(@PathVariable Long id) {
        attendanceService.deleteAttendance(id);
        return "redirect:/admin/attendance";
    }
}

```

EmployeeController.java — com.data.ets.controller

```

package com.data.ets.controller;

import com.data.ets.dto.AttendanceDTO;
import com.data.ets.dto.EmployeeDTO;
import com.data.ets.dto.LeaveDTO;
import com.data.ets.model.Attendance;
import com.data.ets.model.Employee;
import com.data.ets.model.Leave;
import com.data.ets.service.AttendanceService;
import com.data.ets.service.EmployeeService;
import com.data.ets.service.LeaveService;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@Controller
@RequestMapping("/employee")
@RequiredArgsConstructor
public class EmployeeController {
    private final EmployeeService employeeService;
    private final AttendanceService attendanceService;
    private final LeaveService leaveService;

    @GetMapping("/dashboard")
    public String dashboard(Authentication authentication, Model model) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);

        if (employee != null) {
            model.addAttribute("employee", employeeService.convertToDTO(employee));
            model.addAttribute("recentAttendance", attendanceService.getEmployeeAttendance(employee.getId()));
            model.addAttribute("pendingLeaves", leaveService.getEmployeeLeaves(employee.getId()).stream()
                .filter(l -> l.getStatus().equals("PENDING"))
                .toList());
        }

        return "employee/dashboard";
    }

    @GetMapping("/profile")
    public String profile(Authentication authentication, Model model) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);

        if (employee != null) {
            model.addAttribute("employee", employeeService.convertToDTO(employee));
        }

        return "employee/profile";
    }

    // Attendance
    @GetMapping("/attendance")
    public String listAttendance(Authentication authentication, Model model) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);

        if (employee != null) {
            List<AttendanceDTO> attendanceList = attendanceService.getEmployeeAttendance(employee.getId());
            model.addAttribute("attendanceList", attendanceList);
            model.addAttribute("totalPresent", attendanceList.stream().filter(a -> a.getStatus().equals("PRESENT")).count());
            model.addAttribute("totalAbsent", attendanceList.stream().filter(a -> a.getStatus().equals("ABSENT")).count());
            model.addAttribute("totalLate", attendanceList.stream().filter(a -> a.getStatus().equals("LATE")).count());
        }

        return "employee/attendance";
    }

    @PostMapping("/attendance/check-in")
    public String checkIn(Authentication authentication) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);
    }

```

```

        if (employee != null) {
            attendanceService.checkIn(employee.getId());
        }

        return "redirect:/employee/attendance";
    }

    @PostMapping("/attendance/check-out/{attendanceId}")
    public String checkOut(@PathVariable Long attendanceId) {
        attendanceService.checkOut(attendanceId);
        return "redirect:/employee/attendance";
    }

    // Leave Management
    @GetMapping("/leaves")
    public String listLeaves(Authentication authentication, Model model) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);

        if (employee != null) {
            List<LeaveDTO> leaves = leaveService.getEmployeeLeaves(employee.getId());
            model.addAttribute("leaves", leaves);
            model.addAttribute("approved", leaves.stream().filter(l -> l.getStatus().equals("APPROVED")).count());
            model.addAttribute("pending", leaves.stream().filter(l -> l.getStatus().equals("PENDING")).count());
            model.addAttribute("rejected", leaves.stream().filter(l -> l.getStatus().equals("REJECTED")).count());
        }

        return "employee/leaves";
    }

    @GetMapping("/leaves/apply")
    public String applyLeaveForm() {
        return "employee/apply-leave";
    }

    @PostMapping("/leaves/save")
    public String applyLeave(Authentication authentication,
        @RequestParam String leaveType,
        @RequestParam LocalDate startDate,
        @RequestParam LocalDate endDate,
        @RequestParam String reason) {
        String username = authentication.getName();
        Employee employee = employeeService.findAll().stream()
            .filter(e -> e.getUser().getUsername().equals(username))
            .findFirst()
            .orElse(null);

        if (employee != null) {
            Leave leave = Leave.builder()
                .employee(employee)
                .leaveType(leaveType)
                .startDate(startDate)
                .endDate(endDate)
                .reason(reason)
                .status("PENDING")
                .build();
            leaveService.createLeave(leave);
        }

        return "redirect:/employee/leaves";
    }
}

```

LeaveController.java — com.data.ets.controller

```
package com.data.ets.controller;

import com.data.ets.dto.LeaveDTO;
import com.data.ets.service.LeaveService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Controller
@RequestMapping("/admin/leaves")
@RequiredArgsConstructor
public class LeaveController {
    private final LeaveService leaveService;

    @GetMapping
    public String listLeaves(Model model) {
        List<LeaveDTO> leaves = leaveService.getAllLeaves();
        model.addAttribute("leaves", leaves);
        return "admin/leaves/list";
    }

    @GetMapping("/pending")
    public String pendingLeaves(Model model) {
        List<LeaveDTO> pendingLeaves = leaveService.getPendingLeaves();
        model.addAttribute("leaves", pendingLeaves);
        return "admin/leaves/pending";
    }

    @PostMapping("/approve/{id}")
    public String approveLeave(@PathVariable Long id, @RequestParam String approvedBy) {
        leaveService.approveLeave(id, approvedBy);
        return "redirect:/admin/leaves/pending";
    }

    @PostMapping("/reject/{id}")
    public String rejectLeave(@PathVariable Long id, @RequestParam String approvedBy) {
        leaveService.rejectLeave(id, approvedBy);
        return "redirect:/admin/leaves/pending";
    }

    @GetMapping("/delete/{id}")
    public String deleteLeave(@PathVariable Long id) {
        leaveService.deleteLeave(id);
        return "redirect:/admin/leaves";
    }
}
```

CHAPTER 6 : TESTING

The project is validated through unit and integration test setup in Maven, route access verification by role, form validation checks, and end-to-end functional flow testing for attendance and leave operations.

Test Area	Verification	Status
Authentication	Valid and invalid login scenarios	Pass
Authorization	Route access based on role	Pass
Employee CRUD	Create/update/delete/list operations	Pass
Attendance	Check-in/check-out and history flow	Pass
Leave Workflow	Apply, approve, reject lifecycle	Pass

CHAPTER 7 : RESULT AND APPLICATIONS

7.1 Project Results and Outcomes

The final solution successfully delivers all targeted modules and provides a robust base for future enterprise extensions. The platform improves process consistency, decreases manual effort, and increases visibility over employee activities.

Key outcomes include complete CRUD functionality, role-segmented dashboards, secure authentication, clean UI implementation, and maintainable code organization.

7.2 Challenges and Solutions

Challenge	Approach	Outcome
Role-based page isolation	Centralized security configuration and URL policies	Stable permission boundaries
Data consistency across modules	Layered service logic and entity mapping	Predictable workflow behavior
Responsive UI for admin and employee screens	Bootstrap-based shared layout	Consistent cross-page UX
Validation and error feedback	Form binding + validation message handling	Improved user input quality

CHAPTER 8 : CONCLUSION AND FUTURE SCOPE

8.1 Future Enhancements

- Email and in-app notifications for leave decisions.
- Advanced analytics dashboard and PDF/Excel reporting.
- Payroll and biometric integration.
- REST API exposure for mobile clients.
- Cloud deployment with CI/CD and monitoring.

8.2 Conclusion

The Employee Tracking System capstone project demonstrates end-to-end full-stack development with secure architecture and practical business workflows. It satisfies major HR operational requirements and serves as a strong foundation for production-grade expansion.

CHAPTER 9 : REFERENCES AND BIBLIOGRAPHY

- [Spring Boot Official Documentation](#)
- [Spring Security Reference Guide](#)
- [Hibernate ORM Documentation](#)
- [Thymeleaf Documentation](#)
- [MySQL 8.0 Reference Manual](#)
- [Bootstrap 5 Documentation](#)

Prepared for academic capstone submission purposes.